Immutability & Persistent Collections

ersistent Collections · Immutable playlist edits sharing structure · Build with Waqa

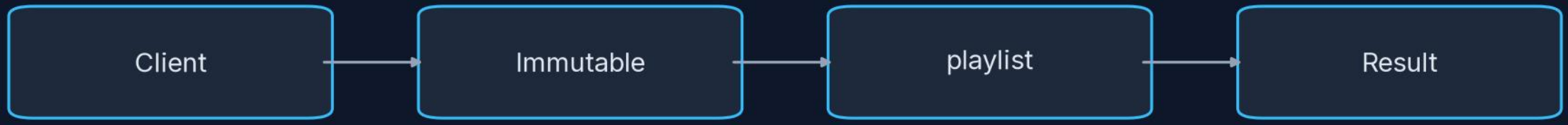
Today's Hook

- Problem: Immutable playlist edits sharing structure
- Pattern: Persistent Collections — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Immutable playlist edits sharing structure
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Persistent Collections: Immutable playlist edits sharing structure

Real-World Scenario

- Running example: Immutable playlist edits sharing structure
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.collections;

import com.buildwithwaqar.patterns.functional.collections.ZazrCollectionsDemo;
import dev.zazr.collection.*;
/** Problem: edit a playlist immutably while sharing structure. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Persistent Collections problem: playlist edits ===");
    var xs = Vector.of(1,2,3);
    System.out.println("orig=" + xs + " doubled=" + ZazrCollectionsDemo.doubleAll(xs));
    System.out.println("max=" + ZazrCollectionsDemo.maxScore(NonEmptyVector.of(7,3,9)));
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.collections;

import com.buildwithwaqar.patterns.functional.collections.ZazrCollectionsDemo;
import dev.zazr.collection.*;
/** Problem: edit a playlist immutably while sharing structure. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Persistent Collections problem: playlist edits ===");
    var xs = Vector.of(1,2,3);
    System.out.println("orig=" + xs + " doubled=" + ZazrCollectionsDemo.doubleAll(xs));
    System.out.println("max=" + ZazrCollectionsDemo.maxScore(NonEmptyVector.of(7,3,9)));
  }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.collections;

import com.buildwithwaqar.patterns.functional.collections.ZazrCollectionsDemo;
import dev.zazr.collection.*;
/** Problem: edit a playlist immutably while sharing structure. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Persistent Collections problem: playlist edits ===");
    var xs = Vector.of(1,2,3);
    System.out.println("orig=" + xs + " doubled=" + ZazrCollectionsDemo.doubleAll(xs));
    System.out.println("max=" + ZazrCollectionsDemo.maxScore(NonEmptyVector.of(7,3,9)));
  }
}
```

Walkthrough Output

```
=== Persistent Collections problem: playlist edits ===
orig=Vector(1, 2, 3) doubled=Vector(2, 4, 6)
max=9
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Persistent Collections.

Related Patterns

- Contrast Persistent Collections with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Persistent Collections → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.